FastAPI Reference

Security Tools

When you need to declare dependencies with OAuth 2 scopes you use `Security()`.

But you still need to define what is the dependable, the callable that you pass as a parameter to `Depends()` or `Security()`.

There are multiple tools that you can use to create those dependables, and they get integrated into OpenAPI so they are shown in the automatic docs UI, they can be used by automatically generated clients and SDKs, etc.

You can import them from `fastapi.security`:

```
from fastapi.security import (
    APIKeyCookie,
    APIKeyHeader,
    APIKeyQuery,
    HTTPAuthorizationCredentials,
    HTTPBasic,
    HTTPBasicCredentials,
    HTTPBearer,
    HTTPDigest,
    OAuth2,
    OAuth2AuthorizationCodeBearer,
    OAuth2PasswordBearer,
    OAuth2PasswordRequestForm,
    OAuth2PasswordRequestFormStrict,
    OpenIdConnect,
    SecurityScopes,
)
```

Read more about them in the FastAPI docs about Security.

API Key Security Schemes

`class` fastapi.security.APIKeyCookie

```
APIKeyCookie(
    *,
    name,
    scheme_name=None,
    description=None,
    auto_error=True
)
```

Bases: `APIKeyBase`

API key authentication using a cookie.

This defines the name of the cookie that should be provided in the request with the API key and integrates that into the OpenAPI documentation. It extracts the key value sent in the cookie automatically and provides it as the dependency result. But it doesn't define how to set that cookie.

Usage

Ask AI

Create an instance object and use that object as the dependency in `Depends()`.

The dependency result will be a string containing the key value.

Example

```
from fastapi import Depends, FastAPI
from fastapi.security import APIKeyCookie

app = FastAPI()

cookie_scheme = APIKeyCookie(name="session")


@app.get("/items/")
async def read_items(session: str = Depends(cookie_scheme)):
    return {"session": session}
```

PARAMETER	DESCRIPTION
`name`	Cookie name. **TYPE:** `str`
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if the cookie is not provided, `APIKeyCookie` will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the cookie is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, in a cookie or in an HTTP Bearer token). **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/security/api_key.py`

```python
def __init__(
    self,
    *,
    name: Annotated[str, Doc("Cookie name.")],
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if the cookie is not provided, `APIKeyCookie` will
            automatically cancel the request and send the client an error.

            If `auto_error` is set to `False`, when the cookie is not available,
            instead of erroring out, the dependency result will be `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, in a cookie or
            in an HTTP Bearer token).
            """
        ),
    ] = True,
):
    super().__init__(
        location=APIKeyIn.cookie,
        name=name,
        scheme_name=scheme_name,
        description=description,
        auto_error=auto_error,
    )
```

attr model `instance-attribute`

```python
model = APIKey(
    **{"in": location}, name=name, description=description
)
```

attr scheme_name `instance-attribute`

```python
scheme_name = scheme_name or __name__
```

attr auto_error `instance-attribute`

```
auto_error = auto_error
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

The WWW-Authenticate header is not standardized for API Key authentication but the HTTP specification requires that an error of `401` "Unauthorized" must include a WWW-Authenticate header.

Ref: https://datatracker.ietf.org/doc/html/rfc9110#name-401-unauthorized

For this, this method sends a custom challenge `APIKey`.

Source code in `fastapi/security/api_key.py`

```
def make_not_authenticated_error(self) -> HTTPException:
    """
    The WWW-Authenticate header is not standardized for API Key authentication but
    the HTTP specification requires that an error of 401 "Unauthorized" must
    include a WWW-Authenticate header.

    Ref: https://datatracker.ietf.org/doc/html/rfc9110#name-401-unauthorized

    For this, this method sends a custom challenge `APIKey`.
    """
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers={"WWW-Authenticate": "APIKey"},
    )
```

meth check_api_key

```
check_api_key(api_key)
```

Source code in `fastapi/security/api_key.py`

```
def check_api_key(self, api_key: str | None) -> str | None:
    if not api_key:
        if self.auto_error:
            raise self.make_not_authenticated_error()
        return None
    return api_key
```

class fastapi.security.APIKeyHeader

```
APIKeyHeader(
    *,
    name,
    scheme_name=None,
    description=None,
    auto_error=True
)
```

Bases: `APIKeyBase`

API key authentication using a header.

This defines the name of the header that should be provided in the request with the API key and integrates that into the OpenAPI documentation. It extracts the key value sent in the header automatically and provides it as the dependency result. But it doesn't define how to send that key to the client.

Usage

Create an instance object and use that object as the dependency in `Depends()`.

The dependency result will be a string containing the key value.

Example

```python
from fastapi import Depends, FastAPI
from fastapi.security import APIKeyHeader

app = FastAPI()

header_scheme = APIKeyHeader(name="x-key")


@app.get("/items/")
async def read_items(key: str = Depends(header_scheme)):
    return {"key": key}
```

PARAMETER	DESCRIPTION
`name`	Header name. **TYPE:** `str`
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if the header is not provided, `APIKeyHeader` will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the header is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, in a header or in an HTTP Bearer token). **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/security/api_key.py`

```python
def __init__(
    self,
    *,
    name: Annotated[str, Doc("Header name.")],
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if the header is not provided, `APIKeyHeader` will
            automatically cancel the request and send the client an error.

            If `auto_error` is set to `False`, when the header is not available,
            instead of erroring out, the dependency result will be `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, in a header or
            in an HTTP Bearer token).
            """
        ),
    ] = True,
):
    super().__init__(
        location=APIKeyIn.header,
        name=name,
        scheme_name=scheme_name,
        description=description,
        auto_error=auto_error,
    )
```

attr model `instance-attribute`

```python
model = APIKey(
    **{"in": location}, name=name, description=description
)
```

attr scheme_name `instance-attribute`

```python
scheme_name = scheme_name or __name__
```

attr auto_error `instance-attribute`

```
auto_error = auto_error
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

The WWW-Authenticate header is not standardized for API Key authentication but the HTTP specification requires that an error of `401` "Unauthorized" must include a WWW-Authenticate header.

Ref: https://datatracker.ietf.org/doc/html/rfc9110#name-401-unauthorized

For this, this method sends a custom challenge `APIKey`.

Source code in `fastapi/security/api_key.py`

```
def make_not_authenticated_error(self) -> HTTPException:
    """
    The WWW-Authenticate header is not standardized for API Key authentication but
    the HTTP specification requires that an error of 401 "Unauthorized" must
    include a WWW-Authenticate header.

    Ref: https://datatracker.ietf.org/doc/html/rfc9110#name-401-unauthorized

    For this, this method sends a custom challenge `APIKey`.
    """
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers={"WWW-Authenticate": "APIKey"},
    )
```

meth check_api_key

```
check_api_key(api_key)
```

Source code in `fastapi/security/api_key.py`

```
def check_api_key(self, api_key: str | None) -> str | None:
    if not api_key:
        if self.auto_error:
            raise self.make_not_authenticated_error()
        return None
    return api_key
```

class fastapi.security.APIKeyQuery

```
APIKeyQuery(
    *,
    name,
    scheme_name=None,
    description=None,
    auto_error=True
)
```

Bases: `APIKeyBase`

API key authentication using a query parameter.

This defines the name of the query parameter that should be provided in the request with the API key and integrates that into the OpenAPI documentation. It extracts the key value sent in the query parameter automatically and provides it as the dependency result. But it doesn't define how to send that API key to the client.

Usage

Create an instance object and use that object as the dependency in `Depends()`.

The dependency result will be a string containing the key value.

Example

```
from fastapi import Depends, FastAPI
from fastapi.security import APIKeyQuery

app = FastAPI()

query_scheme = APIKeyQuery(name="api_key")


@app.get("/items/")
async def read_items(api_key: str = Depends(query_scheme)):
    return {"api_key": api_key}
```

PARAMETER	DESCRIPTION
`name`	Query parameter name. **TYPE:** `str`
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if the query parameter is not provided, `APIKeyQuery` will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the query parameter is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, in a query parameter or in an HTTP Bearer token). **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/security/api_key.py`

```python
def __init__(
    self,
    *,
    name: Annotated[
        str,
        Doc("Query parameter name."),
    ],
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if the query parameter is not provided, `APIKeyQuery` will
            automatically cancel the request and send the client an error.

            If `auto_error` is set to `False`, when the query parameter is not
            available, instead of erroring out, the dependency result will be
            `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, in a query
            parameter or in an HTTP Bearer token).
            """
        ),
    ] = True,
):
    super().__init__(
        location=APIKeyIn.query,
        name=name,
        scheme_name=scheme_name,
        description=description,
        auto_error=auto_error,
    )
```

attr model `instance-attribute`

```python
model = APIKey(
    **{"in": location}, name=name, description=description
)
```

attr scheme_name `instance-attribute`

```python
scheme_name = scheme_name or __name__
```

attr auto_error instance-attribute

```
auto_error = auto_error
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

The WWW-Authenticate header is not standardized for API Key authentication but the HTTP specification requires that an error of 401 "Unauthorized" must include a WWW-Authenticate header.

Ref: https://datatracker.ietf.org/doc/html/rfc9110 # name-401 -unauthorized

For this, this method sends a custom challenge `APIKey`.

Source code in `fastapi/security/api_key.py`

```
def make_not_authenticated_error(self) -> HTTPException:
    """
    The WWW-Authenticate header is not standardized for API Key authentication but
    the HTTP specification requires that an error of 401 "Unauthorized" must
    include a WWW-Authenticate header.

    Ref: https://datatracker.ietf.org/doc/html/rfc9110#name-401-unauthorized

    For this, this method sends a custom challenge `APIKey`.
    """
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers={"WWW-Authenticate": "APIKey"},
    )
```

meth check_api_key

```
check_api_key(api_key)
```

Source code in `fastapi/security/api_key.py`

```
def check_api_key(self, api_key: str | None) -> str | None:
    if not api_key:
        if self.auto_error:
            raise self.make_not_authenticated_error()
        return None
    return api_key
```

HTTP Authentication Schemes

class fastapi.security.HTTPBasic

```
HTTPBasic(
    *,
    scheme_name=None,
    realm=None,
    description=None,
    auto_error=True
```

```
)
```

Bases: `HTTPBase`

HTTP Basic authentication.

Ref: https://datatracker.ietf.org/doc/html/rfc 7 6 1 7

Usage

Create an instance object and use that object as the dependency in `Depends()`.

The dependency result will be an `HTTPBasicCredentials` object containing the `username` and the `password`.

Read more about it in the FastAPI docs for HTTP Basic Auth.

Example

```python
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import HTTPBasic, HTTPBasicCredentials

app = FastAPI()

security = HTTPBasic()


@app.get("/users/me")
def read_current_user(credentials: Annotated[HTTPBasicCredentials, Depends(security)]):
    return {"username": credentials.username, "password": credentials.password}
```

PARAMETER	DESCRIPTION
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`realm`	HTTP Basic authentication realm. **TYPE:** `str | None` **DEFAULT:** `None`
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if the HTTP Basic authentication is not provided (a header), `HTTPBasic` will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the HTTP Basic authentication is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, in HTTP Basic authentication or in an HTTP Bearer token). **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/security/http.py`

```python
def __init__(
    self,
    *,
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    realm: Annotated[
        str | None,
        Doc(
            """
            HTTP Basic authentication realm.
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if the HTTP Basic authentication is not provided (a
            header), `HTTPBasic` will automatically cancel the request and send the
            client an error.

            If `auto_error` is set to `False`, when the HTTP Basic authentication
            is not available, instead of erroring out, the dependency result will
            be `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, in HTTP Basic
            authentication or in an HTTP Bearer token).
            """
        ),
    ] = True,
):
    self.model = HTTPBaseModel(scheme="basic", description=description)
    self.scheme_name = scheme_name or self.__class__.__name__
    self.realm = realm
    self.auto_error = auto_error
```

attr model `instance-attribute`

```python
model = HTTPBase(scheme='basic', description=description)
```

attr scheme_name `instance-attribute`

```python
scheme_name = scheme_name or __name__
```

attr realm instance-attribute

```
realm = realm
```

attr auto_error instance-attribute

```
auto_error = auto_error
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

Source code in `fastapi/security/http.py`

```
def make_not_authenticated_error(self) -> HTTPException:
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers=self.make_authenticate_headers(),
    )
```

meth make_authenticate_headers

```
make_authenticate_headers()
```

Source code in `fastapi/security/http.py`

```
def make_authenticate_headers(self) -> dict[str, str]:
    if self.realm:
        return {"WWW-Authenticate": f'Basic realm="{self.realm}"'}
    return {"WWW-Authenticate": "Basic"}
```

class fastapi.security.HTTPBearer

```
HTTPBearer(
    *,
    bearerFormat=None,
    scheme_name=None,
    description=None,
    auto_error=True
)
```

Bases: `HTTPBase`

HTTP Bearer token authentication.

Usage

Create an instance object and use that object as the dependency in `Depends()`.

The dependency result will be an `HTTPAuthorizationCredentials` object containing the `scheme` and the `credentials`.

Example

```python
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer

app = FastAPI()

security = HTTPBearer()


@app.get("/users/me")
def read_current_user(
    credentials: Annotated[HTTPAuthorizationCredentials, Depends(security)]
):
    return {"scheme": credentials.scheme, "credentials": credentials.credentials}
```

PARAMETER	DESCRIPTION
`bearerFormat`	Bearer token format. **TYPE:** `str | None` **DEFAULT:** `None`
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if the HTTP Bearer token is not provided (in an `Authorization` header), `HTTPBearer` will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the HTTP Bearer token is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, in an HTTP Bearer token or in a cookie). **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/security/http.py`

```python
def __init__(
    self,
    *,
    bearerFormat: Annotated[str | None, Doc("Bearer token format.")] = None,
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if the HTTP Bearer token is not provided (in an
            `Authorization` header), `HTTPBearer` will automatically cancel the
            request and send the client an error.

            If `auto_error` is set to `False`, when the HTTP Bearer token
            is not available, instead of erroring out, the dependency result will
            be `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, in an HTTP
            Bearer token or in a cookie).
            """
        ),
    ] = True,
):
    self.model = HTTPBearerModel(bearerFormat=bearerFormat, description=description)
    self.scheme_name = scheme_name or self.__class__.__name__
    self.auto_error = auto_error
```

attr model `instance-attribute`

```python
model = HTTPBearer(
    bearerFormat=bearerFormat, description=description
)
```

attr scheme_name `instance-attribute`

```python
scheme_name = scheme_name or __name__
```

attr auto_error `instance-attribute`

```python
auto_error = auto_error
```

meth make_authenticate_headers

```
make_authenticate_headers()
```

Source code in `fastapi/security/http.py`

```
def make_authenticate_headers(self) -> dict[str, str]:
    return {"WWW-Authenticate": f"{self.model.scheme.title()}"}
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

Source code in `fastapi/security/http.py`

```
def make_not_authenticated_error(self) -> HTTPException:
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers=self.make_authenticate_headers(),
    )
```

class fastapi.security.HTTPDigest

```
HTTPDigest(
    *, scheme_name=None, description=None, auto_error=True
)
```

Bases: `HTTPBase`

HTTP Digest authentication.

Warning: this is only a stub to connect the components with OpenAPI in FastAPI, but it doesn't implement the full Digest scheme, you would need to subclass it and implement it in your code.

Ref: https://datatracker.ietf.org/doc/html/rfc 7 6 1 6

Usage

Create an instance object and use that object as the dependency in `Depends()`.

The dependency result will be an `HTTPAuthorizationCredentials` object containing the `scheme` and the `credentials`.

Example

```
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import HTTPAuthorizationCredentials, HTTPDigest

app = FastAPI()

security = HTTPDigest()


@app.get("/users/me")
```

```python
def read_current_user(
    credentials: Annotated[HTTPAuthorizationCredentials, Depends(security)]
):
    return {"scheme": credentials.scheme, "credentials": credentials.credentials}
```

PARAMETER	DESCRIPTION
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if the HTTP Digest is not provided, `HTTPDigest` will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the HTTP Digest is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, in HTTP Digest or in a cookie). **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/security/http.py`

```
def __init__(
    self,
    *,
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if the HTTP Digest is not provided, `HTTPDigest` will
            automatically cancel the request and send the client an error.

            If `auto_error` is set to `False`, when the HTTP Digest is not
            available, instead of erroring out, the dependency result will
            be `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, in HTTP
            Digest or in a cookie).
            """
        ),
    ] = True,
):
    self.model = HTTPBaseModel(scheme="digest", description=description)
    self.scheme_name = scheme_name or self.__class__.__name__
    self.auto_error = auto_error
```

attr model `instance-attribute`

```
model = HTTPBase(scheme='digest', description=description)
```

attr scheme_name `instance-attribute`

```
scheme_name = scheme_name or __name__
```

attr auto_error `instance-attribute`

```
auto_error = auto_error
```

meth make_authenticate_headers

```
make_authenticate_headers()
```

Source code in `fastapi/security/http.py`

```
def make_authenticate_headers(self) -> dict[str, str]:
    return {"WWW-Authenticate": f"{self.model.scheme.title()}"}
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

Source code in `fastapi/security/http.py`

```
def make_not_authenticated_error(self) -> HTTPException:
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers=self.make_authenticate_headers(),
    )
```

HTTP Credentials

class fastapi.security.HTTPAuthorizationCredentials

Bases: `BaseModel`

The HTTP authorization credentials in the result of using `HTTPBearer` or `HTTPDigest` in a dependency.

The HTTP authorization header value is split by the first space.

The first part is the `scheme`, the second part is the `credentials`.

For example, in an HTTP Bearer token scheme, the client will send a header like:

```
Authorization: Bearer deadbeef12346
```

In this case:

- `scheme` will have the value `"Bearer"`
- `credentials` will have the value `"deadbeef12346"`

attr scheme `instance-attribute`

```
scheme
```

The HTTP authorization scheme extracted from the header value.

attr credentials `instance-attribute`

```
credentials
```

The HTTP authorization credentials extracted from the header value.

class fastapi.security.HTTPBasicCredentials

Bases: `BaseModel`

The HTTP Basic credentials given as the result of using `HTTPBasic` in a dependency.

Read more about it in the FastAPI docs for HTTP Basic Auth.

attr username `instance-attribute`

```
username
```

The HTTP Basic username.

attr password `instance-attribute`

```
password
```

The HTTP Basic password.

OAuth 2 Authentication

class fastapi.security.OAuth 2

```
OAuth2(
    *,
    flows=OAuthFlows(),
    scheme_name=None,
    description=None,
    auto_error=True
)
```

Bases: `SecurityBase`

This is the base class for OAuth 2 authentication, an instance of it would be used as a dependency. All other OAuth 2 classes inherit from it and customize it for each OAuth 2 flow.

You normally would not create a new class inheriting from it but use one of the existing subclasses, and maybe compose them if you want to support multiple flows.

Read more about it in the FastAPI docs for Security.

PARAMETER	DESCRIPTION
`flows`	The dictionary of OAuth 2 flows. **TYPE:** `OAuthFlows` | `dict[str, dict[str, Any]]` **DEFAULT:** `OAuthFlows()`

PARAMETER	DESCRIPTION
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if no HTTP Authorization header is provided, required for OAuth `2` authentication, it will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the HTTP Authorization header is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, with OAuth `2` or in a cookie). **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/security/oauth2.py`

```python
def __init__(
    self,
    *,
    flows: Annotated[
        OAuthFlowsModel | dict[str, dict[str, Any]],
        Doc(
            """
            The dictionary of OAuth2 flows.
            """
        ),
    ] = OAuthFlowsModel(),
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if no HTTP Authorization header is provided, required for
            OAuth2 authentication, it will automatically cancel the request and
            send the client an error.

            If `auto_error` is set to `False`, when the HTTP Authorization header
            is not available, instead of erroring out, the dependency result will
            be `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, with OAuth2
            or in a cookie).
            """
        ),
    ] = True,
):
    self.model = OAuth2Model(
        flows=cast(OAuthFlowsModel, flows), description=description
    )
    self.scheme_name = scheme_name or self.__class__.__name__
    self.auto_error = auto_error
```

attr model `instance-attribute`

```python
model = OAuth2(
    flows=cast(OAuthFlows, flows), description=description
)
```

attr scheme_name `instance-attribute`

```
scheme_name = scheme_name or __name__
```

attr auto_error `instance-attribute`

```
auto_error = auto_error
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

The OAuth 2 specification doesn't define the challenge that should be used, because a `Bearer` token is not really the only option to authenticate.

But declaring any other authentication challenge would be application-specific as it's not defined in the specification.

For practical reasons, this method uses the `Bearer` challenge by default, as it's probably the most common one.

If you are implementing an OAuth 2 authentication scheme other than the provided ones in FastAPI (based on bearer tokens), you might want to override this.

Ref: https://datatracker.ietf.org/doc/html/rfc 6 7 4 9

Source code in `fastapi/security/oauth2.py`

```
def make_not_authenticated_error(self) -> HTTPException:
    """
    The OAuth 2 specification doesn't define the challenge that should be used,
    because a `Bearer` token is not really the only option to authenticate.

    But declaring any other authentication challenge would be application-specific
    as it's not defined in the specification.

    For practical reasons, this method uses the `Bearer` challenge by default, as
    it's probably the most common one.

    If you are implementing an OAuth2 authentication scheme other than the provided
    ones in FastAPI (based on bearer tokens), you might want to override this.

    Ref: https://datatracker.ietf.org/doc/html/rfc6749
    """
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers={"WWW-Authenticate": "Bearer"},
    )
```

class fastapi.security.OAuth 2 AuthorizationCodeBearer

```
OAuth2AuthorizationCodeBearer(
    authorizationUrl,
    tokenUrl,
    refreshUrl=None,
    scheme_name=None,
    scopes=None,
    description=None,
    auto_error=True,
)
```

Bases: `OAuth2`

OAuth 2 flow for authentication using a bearer token obtained with an OAuth 2 code flow. An instance of it would be used as a dependency.

PARAMETER	DESCRIPTION
`tokenUrl`	The URL to obtain the OAuth 2 token. **TYPE:** `str`
`refreshUrl`	The URL to refresh the token and obtain a new one. **TYPE:** `str | None` **DEFAULT:** `None`
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`scopes`	The OAuth 2 scopes that would be required by the *path operations* that use this dependency. **TYPE:** `dict[str, str] | None` **DEFAULT:** `None`
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if no HTTP Authorization header is provided, required for OAuth 2 authentication, it will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the HTTP Authorization header is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, with OAuth 2 or in a cookie). **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/security/oauth2.py`

```python
def __init__(
    self,
    authorizationUrl: str,
    tokenUrl: Annotated[
        str,
        Doc(
            """
            The URL to obtain the OAuth2 token.
            """
        ),
    ],
    refreshUrl: Annotated[
        str | None,
        Doc(
            """
            The URL to refresh the token and obtain a new one.
            """
        ),
    ] = None,
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    scopes: Annotated[
        dict[str, str] | None,
        Doc(
            """
            The OAuth2 scopes that would be required by the *path operations* that
            use this dependency.
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if no HTTP Authorization header is provided, required for
            OAuth2 authentication, it will automatically cancel the request and
            send the client an error.

            If `auto_error` is set to `False`, when the HTTP Authorization header
            is not available, instead of erroring out, the dependency result will
            be `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, with OAuth2
            or in a cookie).
            """
        ),
    ] = True,
):
    if not scopes:
        scopes = {}
    flows = OAuthFlowsModel(
```

```
        authorizationCode=cast(
            Any,
            {
                "authorizationUrl": authorizationUrl,
                "tokenUrl": tokenUrl,
                "refreshUrl": refreshUrl,
                "scopes": scopes,
            },
        )
    )
    super().__init__(
        flows=flows,
        scheme_name=scheme_name,
        description=description,
        auto_error=auto_error,
    )
```

attr model `instance-attribute`

```
model = OAuth2(
    flows=cast(OAuthFlows, flows), description=description
)
```

attr scheme_name `instance-attribute`

```
scheme_name = scheme_name or __name__
```

attr auto_error `instance-attribute`

```
auto_error = auto_error
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

The OAuth 2 specification doesn't define the challenge that should be used, because a `Bearer` token is not really the only option to authenticate.

But declaring any other authentication challenge would be application-specific as it's not defined in the specification.

For practical reasons, this method uses the `Bearer` challenge by default, as it's probably the most common one.

If you are implementing an OAuth 2 authentication scheme other than the provided ones in FastAPI (based on bearer tokens), you might want to override this.

Ref: https://datatracker.ietf.org/doc/html/rfc 6 7 4 9

Source code in `fastapi/security/oauth2.py`

```python
def make_not_authenticated_error(self) -> HTTPException:
    """
    The OAuth 2 specification doesn't define the challenge that should be used,
    because a `Bearer` token is not really the only option to authenticate.

    But declaring any other authentication challenge would be application-specific
    as it's not defined in the specification.

    For practical reasons, this method uses the `Bearer` challenge by default, as
    it's probably the most common one.

    If you are implementing an OAuth2 authentication scheme other than the provided
    ones in FastAPI (based on bearer tokens), you might want to override this.

    Ref: https://datatracker.ietf.org/doc/html/rfc6749
    """
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers={"WWW-Authenticate": "Bearer"},
    )
```

`class` fastapi.security.OAuth 2 PasswordBearer

```python
OAuth2PasswordBearer(
    tokenUrl,
    scheme_name=None,
    scopes=None,
    description=None,
    auto_error=True,
    refreshUrl=None,
)
```

Bases: `OAuth2`

OAuth 2 flow for authentication using a bearer token obtained with a password. An instance of it would be used as a dependency.

Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer.

PARAMETER	DESCRIPTION
`tokenUrl`	The URL to obtain the OAuth 2 token. This would be the *path operation* that has `OAuth2PasswordRequestForm` as a dependency. Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `str`
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`scopes`	The OAuth 2 scopes that would be required by the *path operations* that use this dependency. Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `dict[str, str] | None` **DEFAULT:** `None`

PARAMETER	DESCRIPTION
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if no HTTP Authorization header is provided, required for OAuth `2` authentication, it will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the HTTP Authorization header is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, with OAuth `2` or in a cookie). **TYPE:** `bool` **DEFAULT:** `True`
`refreshUrl`	The URL to refresh the token and obtain a new one. **TYPE:** `str | None` **DEFAULT:** `None`

Source code in `fastapi/security/oauth2.py`

```
def __init__(
    self,
    tokenUrl: Annotated[
        str,
        Doc(
            """
            The URL to obtain the OAuth2 token. This would be the *path operation*
            that has `OAuth2PasswordRequestForm` as a dependency.

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
            """
        ),
    ],
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    scopes: Annotated[
        dict[str, str] | None,
        Doc(
            """
            The OAuth2 scopes that would be required by the *path operations* that
            use this dependency.

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if no HTTP Authorization header is provided, required for
            OAuth2 authentication, it will automatically cancel the request and
            send the client an error.

            If `auto_error` is set to `False`, when the HTTP Authorization header
            is not available, instead of erroring out, the dependency result will
            be `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, with OAuth2
            or in a cookie).
            """
        ),
    ] = True,
    refreshUrl: Annotated[
        str | None,
        Doc(
            """
```

```
            The URL to refresh the token and obtain a new one.
            """
        ),
    ] = None,
):
    if not scopes:
        scopes = {}
    flows = OAuthFlowsModel(
        password=cast(
            Any,
            {
                "tokenUrl": tokenUrl,
                "refreshUrl": refreshUrl,
                "scopes": scopes,
            },
        )
    )
    super().__init__(
        flows=flows,
        scheme_name=scheme_name,
        description=description,
        auto_error=auto_error,
    )
```

attr model `instance-attribute`

```
model = OAuth2(
    flows=cast(OAuthFlows, flows), description=description
)
```

attr scheme_name `instance-attribute`

```
scheme_name = scheme_name or __name__
```

attr auto_error `instance-attribute`

```
auto_error = auto_error
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

The OAuth 2 specification doesn't define the challenge that should be used, because a `Bearer` token is not really the only option to authenticate.

But declaring any other authentication challenge would be application-specific as it's not defined in the specification.

For practical reasons, this method uses the `Bearer` challenge by default, as it's probably the most common one.

If you are implementing an OAuth 2 authentication scheme other than the provided ones in FastAPI (based on bearer tokens), you might want to override this.

Ref: https://datatracker.ietf.org/doc/html/rfc 6 7 4 9

Source code in `fastapi/security/oauth2.py`

```python
def make_not_authenticated_error(self) -> HTTPException:
    """
    The OAuth 2 specification doesn't define the challenge that should be used,
    because a `Bearer` token is not really the only option to authenticate.

    But declaring any other authentication challenge would be application-specific
    as it's not defined in the specification.

    For practical reasons, this method uses the `Bearer` challenge by default, as
    it's probably the most common one.

    If you are implementing an OAuth2 authentication scheme other than the provided
    ones in FastAPI (based on bearer tokens), you might want to override this.

    Ref: https://datatracker.ietf.org/doc/html/rfc6749
    """
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers={"WWW-Authenticate": "Bearer"},
    )
```

OAuth 2 Password Form

class fastapi.security.OAuth 2 PasswordRequestForm

```python
OAuth2PasswordRequestForm(
    *,
    grant_type=None,
    username,
    password,
    scope="",
    client_id=None,
    client_secret=None
)
```

This is a dependency class to collect the `username` and `password` as form data for an OAuth 2 password flow.

The OAuth 2 specification dictates that for a password flow the data should be collected using form data (instead of JSON) and that it should have the specific fields `username` and `password`.

All the initialization parameters are extracted from the request.

Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer.

Example

```python
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordRequestForm

app = FastAPI()


@app.post("/login")
def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]):
    data = {}
    data["scopes"] = []
    for scope in form_data.scopes:
        data["scopes"].append(scope)
    if form_data.client_id:
```

```
        data["client_id"] = form_data.client_id
    if form_data.client_secret:
        data["client_secret"] = form_data.client_secret
    return data
```

Note that for OAuth 2 the scope `items:read` is a single scope in an opaque string. You could have custom internal logic to separate it by colon characters (`:`) or similar, and get the two parts `items` and `read`. Many applications do that to group and organize permissions, you could do it as well in your application, just know that it is application specific, it's not part of the specification.

PARAMETER	DESCRIPTION
`grant_type`	The OAuth 2 spec says it is required and MUST be the fixed string "password". Nevertheless, this dependency class is permissive and allows not passing it. If you want to enforce it, use instead the `OAuth2PasswordRequestFormStrict` dependency. Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `str | None` **DEFAULT:** `None`
`username`	`username` string. The OAuth 2 spec requires the exact field name `username`. Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `str`
`password`	`password` string. The OAuth 2 spec requires the exact field name `password`. Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `str`
`scope`	A single string with actually several scopes separated by spaces. Each scope is also a string. For example, a single string with: ```python "items:read items:write users:read profile openid" ``` would represent the scopes: • `items:read` • `items:write` • `users:read` • `profile` • `openid` Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `str` **DEFAULT:** `''`
`client_id`	If there's a `client_id`, it can be sent as part of the form fields. But the OAuth 2 specification recommends sending the `client_id` and `client_secret` (if any) using HTTP Basic auth. **TYPE:** `str | None` **DEFAULT:** `None`
`client_secret`	If there's a `client_password` (and a `client_id`), they can be sent as part of the form fields. But the OAuth 2 specification recommends sending the `client_id` and `client_secret` (if any) using HTTP Basic auth.

PARAMETER	DESCRIPTION
	TYPE: `str | None` **DEFAULT:** `None`

Source code in `fastapi/security/oauth2.py`

```
def __init__(
    self,
    *,
    grant_type: Annotated[
        str | None,
        Form(pattern="^password$"),
        Doc(
            """
            The OAuth2 spec says it is required and MUST be the fixed string
            "password". Nevertheless, this dependency class is permissive and
            allows not passing it. If you want to enforce it, use instead the
            `OAuth2PasswordRequestFormStrict` dependency.

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
            """
        ),
    ] = None,
    username: Annotated[
        str,
        Form(),
        Doc(
            """
            `username` string. The OAuth2 spec requires the exact field name
            `username`.

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
            """
        ),
    ],
    password: Annotated[
        str,
        Form(json_schema_extra={"format": "password"}),
        Doc(
            """
            `password` string. The OAuth2 spec requires the exact field name
            `password`.

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
            """
        ),
    ],
    scope: Annotated[
        str,
        Form(),
        Doc(
            """
            A single string with actually several scopes separated by spaces. Each
            scope is also a string.

            For example, a single string with:

            ```python
            "items:read items:write users:read profile openid"
            ```

            would represent the scopes:

            * `items:read`
            * `items:write`
            * `users:read`
            * `profile`
            * `openid`

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
```

```
            """
        ),
    ] = "",
    client_id: Annotated[
        str | None,
        Form(),
        Doc(
            """
            If there's a `client_id`, it can be sent as part of the form fields.
            But the OAuth2 specification recommends sending the `client_id` and
            `client_secret` (if any) using HTTP Basic auth.
            """
        ),
    ] = None,
    client_secret: Annotated[
        str | None,
        Form(json_schema_extra={"format": "password"}),
        Doc(
            """
            If there's a `client_password` (and a `client_id`), they can be sent
            as part of the form fields. But the OAuth2 specification recommends
            sending the `client_id` and `client_secret` (if any) using HTTP Basic
            auth.
            """
        ),
    ] = None,
):
    self.grant_type = grant_type
    self.username = username
    self.password = password
    self.scopes = scope.split()
    self.client_id = client_id
    self.client_secret = client_secret
```

attr grant_type `instance-attribute`

```
grant_type = grant_type
```

attr username `instance-attribute`

```
username = username
```

attr password `instance-attribute`

```
password = password
```

attr scopes `instance-attribute`

```
scopes = split()
```

attr client_id `instance-attribute`

```
client_id = client_id
```

attr client_secret `instance-attribute`

```
client_secret = client_secret
```

class fastapi.security.OAuth2PasswordRequestFormStrict

```
OAuth2PasswordRequestFormStrict(
    grant_type,
    username,
    password,
    scope="",
    client_id=None,
    client_secret=None,
)
```

Bases: `OAuth2PasswordRequestForm`

This is a dependency class to collect the `username` and `password` as form data for an OAuth 2 password flow.

The OAuth 2 specification dictates that for a password flow the data should be collected using form data (instead of JSON) and that it should have the specific fields `username` and `password`.

All the initialization parameters are extracted from the request.

The only difference between `OAuth2PasswordRequestFormStrict` and `OAuth2PasswordRequestForm` is that `OAuth2PasswordRequestFormStrict` requires the client to send the form field `grant_type` with the value `"password"`, which is required in the OAuth 2 specification (it seems that for no particular reason), while for `OAuth2PasswordRequestForm` `grant_type` is optional.

Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer.

Example

```
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.security import OAuth2PasswordRequestForm

app = FastAPI()


@app.post("/login")
def login(form_data: Annotated[OAuth2PasswordRequestFormStrict, Depends()]):
    data = {}
    data["scopes"] = []
    for scope in form_data.scopes:
        data["scopes"].append(scope)
    if form_data.client_id:
        data["client_id"] = form_data.client_id
    if form_data.client_secret:
        data["client_secret"] = form_data.client_secret
    return data
```

Note that for OAuth 2 the scope `items:read` is a single scope in an opaque string. You could have custom internal logic to separate it by colon characters (`:`) or similar, and get the two parts `items` and `read`. Many applications do that to group and organize permissions, you could do it as well in your application, just know that it is application specific, it's not part of the specification.

the OAuth 2 spec says it is required and MUST be the fixed string "password".

This dependency is strict about it. If you want to be permissive, use instead the OAuth 2 PasswordRequestForm dependency class.

username: username string. The OAuth 2 spec requires the exact field name "username". password: password string. The OAuth 2 spec requires the exact field name "password". scope: Optional string. Several scopes (each one a string)

separated by spaces. E.g. "items:read items:write users:read profile openid" client_id: optional string. OAuth 2 recommends sending the client_id and client_secret (if any) using HTTP Basic auth, as: client_id:client_secret client_secret: optional string. OAuth 2 recommends sending the client_id and client_secret (if any) using HTTP Basic auth, as: client_id:client_secret

PARAMETER	DESCRIPTION
`grant_type`	The OAuth 2 spec says it is required and MUST be the fixed string "password". This dependency is strict about it. If you want to be permissive, use instead the `OAuth2PasswordRequestForm` dependency class. Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `str`
`username`	`username` string. The OAuth 2 spec requires the exact field name `username`. Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `str`
`password`	`password` string. The OAuth 2 spec requires the exact field name `password`. Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `str`
`scope`	A single string with actually several scopes separated by spaces. Each scope is also a string. For example, a single string with: ```python "items:read items:write users:read profile openid" ``` would represent the scopes: • `items:read` • `items:write` • `users:read` • `profile` • `openid` Read more about it in the FastAPI docs for Simple OAuth 2 with Password and Bearer. **TYPE:** `str` **DEFAULT:** `''`
`client_id`	If there's a `client_id`, it can be sent as part of the form fields. But the OAuth 2 specification recommends sending the `client_id` and `client_secret` (if any) using HTTP Basic auth. **TYPE:** `str | None` **DEFAULT:** `None`
`client_secret`	If there's a `client_password` (and a `client_id`), they can be sent as part of the form fields. But the OAuth 2 specification recommends sending the `client_id` and `client_secret` (if any) using HTTP Basic auth. **TYPE:** `str | None` **DEFAULT:** `None`

Source code in `fastapi/security/oauth2.py`

```
def __init__(
    self,
    grant_type: Annotated[
        str,
        Form(pattern="^password$"),
        Doc(
            """
            The OAuth2 spec says it is required and MUST be the fixed string
            "password". This dependency is strict about it. If you want to be
            permissive, use instead the `OAuth2PasswordRequestForm` dependency
            class.

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
            """
        ),
    ],
    username: Annotated[
        str,
        Form(),
        Doc(
            """
            `username` string. The OAuth2 spec requires the exact field name
            `username`.

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
            """
        ),
    ],
    password: Annotated[
        str,
        Form(),
        Doc(
            """
            `password` string. The OAuth2 spec requires the exact field name
            `password`.

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
            """
        ),
    ],
    scope: Annotated[
        str,
        Form(),
        Doc(
            """
            A single string with actually several scopes separated by spaces. Each
            scope is also a string.

            For example, a single string with:

            ```python
            "items:read items:write users:read profile openid"
            ```

            would represent the scopes:

            * `items:read`
            * `items:write`
            * `users:read`
            * `profile`
            * `openid`

            Read more about it in the
            [FastAPI docs for Simple OAuth2 with Password and Bearer]
(https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/).
            """
```

```
        ),
    ] = "",
    client_id: Annotated[
        str | None,
        Form(),
        Doc(
            """
            If there's a `client_id`, it can be sent as part of the form fields.
            But the OAuth2 specification recommends sending the `client_id` and
            `client_secret` (if any) using HTTP Basic auth.
            """
        ),
    ] = None,
    client_secret: Annotated[
        str | None,
        Form(),
        Doc(
            """
            If there's a `client_password` (and a `client_id`), they can be sent
            as part of the form fields. But the OAuth2 specification recommends
            sending the `client_id` and `client_secret` (if any) using HTTP Basic
            auth.
            """
        ),
    ] = None,
):
    super().__init__(
        grant_type=grant_type,
        username=username,
        password=password,
        scope=scope,
        client_id=client_id,
        client_secret=client_secret,
    )
```

attr grant_type `instance-attribute`

```
grant_type = grant_type
```

attr username `instance-attribute`

```
username = username
```

attr password `instance-attribute`

```
password = password
```

attr scopes `instance-attribute`

```
scopes = split()
```

attr client_id `instance-attribute`

```
client_id = client_id
```

attr client_secret `instance-attribute`

```
client_secret = client_secret
```

OAuth 2 Security Scopes in Dependencies

class fastapi.security.SecurityScopes

```
SecurityScopes(scopes=None)
```

This is a special class that you can define in a parameter in a dependency to obtain the OAuth 2 scopes required by all the dependencies in the same chain.

This way, multiple dependencies can have different scopes, even when used in the same *path operation*. And with this, you can access all the scopes required in all those dependencies in a single place.

Read more about it in the FastAPI docs for OAuth 2 scopes.

PARAMETER	DESCRIPTION
`scopes`	This will be filled by FastAPI. **TYPE:** `list[str] | None` **DEFAULT:** `None`

Source code in `fastapi/security/oauth2.py`

```
def __init__(
    self,
    scopes: Annotated[
        list[str] | None,
        Doc(
            """
            This will be filled by FastAPI.
            """
        ),
    ] = None,
):
    self.scopes: Annotated[
        list[str],
        Doc(
            """
            The list of all the scopes required by dependencies.
            """
        ),
    ] = scopes or []
    self.scope_str: Annotated[
        str,
        Doc(
            """
            All the scopes required by all the dependencies in a single string
            separated by spaces, as defined in the OAuth2 specification.
            """
        ),
    ] = " ".join(self.scopes)
```

attr scopes `instance-attribute`

```
scopes = scopes or []
```

The list of all the scopes required by dependencies.

attr scope_str `instance-attribute`

```
scope_str = join(scopes)
```

All the scopes required by all the dependencies in a single string separated by spaces, as defined in the OAuth 2 specification.

OpenID Connect

class fastapi.security.OpenIdConnect

```
OpenIdConnect(
    *,
    openIdConnectUrl,
    scheme_name=None,
    description=None,
    auto_error=True
)
```

Bases: `SecurityBase`

OpenID Connect authentication class. An instance of it would be used as a dependency.

Warning: this is only a stub to connect the components with OpenAPI in FastAPI, but it doesn't implement the full OpenIdConnect scheme, for example, it doesn't use the OpenIDConnect URL. You would need to subclass it and implement it in your code.

PARAMETER	DESCRIPTION
`openIdConnectUrl`	The OpenID Connect URL. **TYPE:** `str`
`scheme_name`	Security scheme name. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`description`	Security scheme description. It will be included in the generated OpenAPI (e.g. visible at `/docs`). **TYPE:** `str | None` **DEFAULT:** `None`
`auto_error`	By default, if no HTTP Authorization header is provided, required for OpenID Connect authentication, it will automatically cancel the request and send the client an error. If `auto_error` is set to `False`, when the HTTP Authorization header is not available, instead of erroring out, the dependency result will be `None`. This is useful when you want to have optional authentication. It is also useful when you want to have authentication that can be provided in one of multiple optional ways (for example, with OpenID Connect or in a cookie). **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/security/open_id_connect_url.py`

```python
def __init__(
    self,
    *,
    openIdConnectUrl: Annotated[
        str,
        Doc(
            """
        The OpenID Connect URL.
        """
        ),
    ],
    scheme_name: Annotated[
        str | None,
        Doc(
            """
            Security scheme name.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    description: Annotated[
        str | None,
        Doc(
            """
            Security scheme description.

            It will be included in the generated OpenAPI (e.g. visible at `/docs`).
            """
        ),
    ] = None,
    auto_error: Annotated[
        bool,
        Doc(
            """
            By default, if no HTTP Authorization header is provided, required for
            OpenID Connect authentication, it will automatically cancel the request
            and send the client an error.

            If `auto_error` is set to `False`, when the HTTP Authorization header
            is not available, instead of erroring out, the dependency result will
            be `None`.

            This is useful when you want to have optional authentication.

            It is also useful when you want to have authentication that can be
            provided in one of multiple optional ways (for example, with OpenID
            Connect or in a cookie).
            """
        ),
    ] = True,
):
    self.model = OpenIdConnectModel(
        openIdConnectUrl=openIdConnectUrl, description=description
    )
    self.scheme_name = scheme_name or self.__class__.__name__
    self.auto_error = auto_error
```

attr model `instance-attribute`

```python
model = OpenIdConnect(
    openIdConnectUrl=openIdConnectUrl,
    description=description,
)
```

attr scheme_name `instance-attribute`

```
scheme_name = scheme_name or __name__
```

attr auto_error `instance-attribute`

```
auto_error = auto_error
```

meth make_not_authenticated_error

```
make_not_authenticated_error()
```

Source code in `fastapi/security/open_id_connect_url.py`

```
def make_not_authenticated_error(self) -> HTTPException:
    return HTTPException(
        status_code=HTTP_401_UNAUTHORIZED,
        detail="Not authenticated",
        headers={"WWW-Authenticate": "Bearer"},
    )
```